Class notes:

1. It's okay to implement only minimization of 'wait'.
2. Acceleration is welcome.
3. It's okay to implement only acceleration, not deceleration.

IDEA: make an 'energy saved' wrapper.

23.06.25

So, we decided to settle for acceleration. Acceleration requires speed to be implemented. Let's get back down to it. Now, I'm prototyping in a very simple manner:

a. Let's get 'atom' atoms in the test encoding be produced by a choice rule, like in a real encoding:

```
1{action_atom(MOMENT, ATTR) : attr(ATTR)}1 :- moment(MOMENT).
```

Good. Now let's try to make them be separated from each other according to the 'speed' fact that is passed in. For this, we'll need to create 'action moments' range again:

```
action_moment(ACTION_MOMENT) :- ACTION_MOMENT = 0..MAX_TIME,
                                all_time(MAX_TIME),
                                speed(MOM_PER_POS),
                                R = ACTION_MOMENT - (ACTION_MOMENT / MOM_PER_POS) * MOM_PER_POS,
                                R == 0.

1{action_atom(ACTION_MOMENT, ATTR) : attr(ATTR)}1 :- action_moment(ACTION_MOMENT).
```

I seem to have managed to replicate the necessary behaviour:

```
Answer: 1
atom(1,b) atom(3,a) atom(0,b) atom(2,a)
Answer: 2
atom(1,c) atom(3,a) atom(0,c) atom(2,a)
Answer: 3
atom(1,d) atom(3,a) atom(0,d) atom(2,a)
Answer: 4
atom(1,b) atom(3,b) atom(0,b) atom(2,b)
Answer: 5
atom(1,d) atom(3,b) atom(0,d) atom(2,b)
Answer: 6
atom(1,c) atom(3,b) atom(0,c) atom(2,b)
Answer: 7
atom(1,b) atom(3,c) atom(0,b) atom(2,c)
Answer: 8
atom(1,c) atom(3,c) atom(0,c) atom(2,c)
```

We see that the 'dwelling' steps repeat their preceding 'action' steps. **That's exactly what we wanted.**

Testing with longer time:

```
Answer: 60
atom(1,a) atom(3,b) atom(5,a) atom(0,a) atom(2,b) atom(4,a)
Answer: 61
atom(1,a) atom(3,c) atom(5,c) atom(0,a) atom(2,c) atom(4,c)
Answer: 62
atom(1,a) atom(3,c) atom(5,b) atom(0,a) atom(2,c) atom(4,b)
Answer: 63
atom(1,a) atom(3,c) atom(5,a) atom(0,a) atom(2,c) atom(4,a)
Answer: 64
atom(1,a) atom(3,c) atom(5,d) atom(0,a) atom(2,c) atom(4,d)
```

Yes, this is correct again.

Let's test with different speeds:

Breaks a little: the tale of the last action atom is missing. Because of this rule most likely:

```
atom(MOMENT, ATTR) :- action_atom(MOMENT-1, ATTR), intermediate_missing(MOMENT).
```

It only checks the immediately preceding missing atom, while it should check for any (in a reasonable scope) length:

```
atom(MOMENT, ATTR) :- action_atom(MOMENT - DIFF_TO_CHECK, ATTR), intermediate_missing(MOMENT), diff_to_check(DIFF_TO_CHECK).
```

Now the normal rule goes over all the preceding atoms defined by the speed. Let's test again. The correct behaviour would be:

The slower the speed, the longer the periods of the same attribute - in a sense, the longer the 'wave length'.

Edge case: speed is lower than the moments of time given - then must be 'unsatisfiable' OR just a piece of the distance (can't predict now how it will work out). Need to test for this.

Total time: 6

Speed	**Result**
1	atom(0,a) atom(1,b) atom(2,c) atom(3,b) atom(4,c) atom(5,a)
1/2	atom(1,a) atom(3,c) atom(5,d) atom(0,a) atom(2,c) atom(4,d)
1/3	atom(1,a) atom(4,d) atom(2,a) atom(5,d) atom(0,a) atom(3,d)
1/6	atom(1,b) atom(2,b) atom(3,b) atom(4,b) atom(5,b) atom(0,b)

INTUITION: there must be a rule about discretizing speed based on the total time given. It seems simple: **speed must be such that the total time is divisible by it without remainder**.

What would be the next step for me?

a. Keep experimenting in the simple isolated encoding.

b. Try to apply what I've arrived at in mine.

c. Try to apply the same logic in Jonas' encoding.

Since I don't see what else to implement in the simplified case, I will try to transfer the speed logic exactly into my encoding. **Once it works,** I'll return to the simple case and try to implement **acceleration**.

Also after I manage to implement speed in mine, I will guide Jonas through implementing the same (if he hasn't arrived at a better solution) in his code.

So, now, let's transit from my simplified encoding into my full pathfinding model:

```
intermediate_missing(TRAIN, MOMENT) :- every_moment(TRAIN, MOMENT),
not action_moment(TRAIN, ACTION_MOMENT), MOMENT == ACTION_MOMENT.
```

The only difference with the simple case logic is that we bind the missing moment to a particular train.

Let's also add the scope check defined by the speed:

```
intermediate_missing(TRAIN, MOMENT),
action_transition(TRAIN, MOMENT-MARGIN, FROM_POS, IN_DIR, ACTION, TO_POS, OUT_DIR),
margin(MARGIN).
```

Done. It seems that it's all that's necessary. Now, let's test this (EXPECTATION: fails for any speed except 1). Yes, **fails for anything but 1**. As usual, I should expect the problem to be sitting in something really simple.

HYPOTHESIS: some of the constraints correctly working with speed of 1 prohibit dwelling.

ALTERNATIVE PATH: develop the prototype into a very bacic train movement; only use the straight track type.

Yes, I'll better go for this. Make the simplest environment possible and see what could prevent speed from work.

Maybe the margin was calculated incorrectly.

25.06.25

We've exchanged on our progress with speed implementation with Jonas. It looks good: we've both tried something. Conceptually we are on the right track. My prototype of 'wavelength speed' works well. The plan now is to try to make it work with actual trains in either / all of three versions:

a. In my original working encoding
b. In Jonas' working encoding
c. In a separate simple encoding

Let's start with extending the logic of the simplest one to imitate train movement - the principle must be the **Occam's razor**: just do the bare minimum to prove the concept, cut off all the fluff (every detail from the 'big' encoding must prove that it's not fluff to be implemented).

Element	**Reasoning**	**Implementation: yes, no or minimized**
train(ID)	Train ids are necessary to differentiate between trains, i.e. when we have many of them. Since in the speed prototype we have only one train, we don't need any indicator.	No.

Tracks with different types	Could be necessary if we want to implement actual turns. Let's	Implement a 1d coordinate track.
		`cell(0).`
		`cell(1).`
		`cell(2).`
		`cell(2).`
		`cell(3).`
		`cell(4).`
		`cell(5).`
		But we won't see the frequency of changing actions then. What's the next complication that's still very simple? A naive thing: let the train do ANY action in any cell, we just care about frequency. Feels pointless.

No, this seems to be a bad path, since I'm building the whole logic from scratch. It's better to adapt / decrease the from the big encoding.

I'm totally lost again. The concept seems clear, but I need to expand it onto actual train movement. Let's return to my big encoding with speed and try to remove constraints one by one

to see if I manage to make it run. The hypothesis is that some of them interfere with the new logic. Didn't help. Switching off more. Yes, switching off all the constraints but the first one gave the result. Let's explore the output:

```
transition(1,1,6,north,wait,6,north)
transition(1,3,6,north,wait,6,north)
transition(1,0,6,north,wait,6,north)
transition(1,2,6,north,wait,6,north)
```

Of course, it's totally broken, but it's because most of the constraints are off. Let's switch them one by one.

DISCOVERY: it breaks down at 'arriving at the destination' constraint. Let's explore the output:

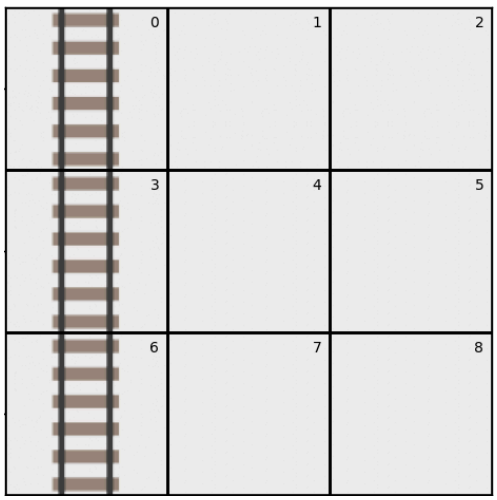
```
transition(1,0,6,north,wait,6,north)
transition(1,1,6,north,wait,6,north)
transition(1,2,6,north,wait,6,north)
transition(1,3,6,north,wait,6,north)
```

Why does the train wait all the time? For the prototype, let's cancel the rule that allows us to wait at the starting point.

26.06.25

Okay, let's keep on this way - turning on and off and modifying the constraints. Removed all paths except for the vertical ones - 32800.

Let's also remove everything possible from the encoding only leaving the parts that are necessary to retain the basic movement. Let's look at this simple map again:



The movement must be: 6 => 3 => 0.

Then, there's the right answer here with this stripped-down encoding:

```
Answer: 1
transition(1,0,6,north,move_forward,3,north) transition(1,2,0,south,move_forward,3,south)
Answer: 2
transition(1,0,6,north,move_forward,3,north) transition(1,2,3,south,move_forward,6,south)
Answer: 3
transition(1,0,6,north,move_forward,3,north) transition(1,2,6,north,move_forward,3,north)
Answer: 4
transition(1,0,6,north,move_forward,3,north) transition(1,2,3,north,move_forward,0,north)
```

Now, let's try to change the speed (and the arrival time accordingly). No! It was already set to ½!

The behaviour is strange. Let's keep experimenting.

Possible reason for the strange output: 'dummy_transition' atoms are not converted into action(), neither are they shown in the output. Let's display both real and dummy transitions and see what we have systematically:

Speed, Travel Time	**Answer Set**	**Commentary**
1, 2	`transition(1,0,6,north,move_forward,3,north)` `transition(1,1,3,north,move_forward,0,north)`	No dummy transitions appear. The train arrives at the destination. Interestingly, it arrives there *without the constraint* necessitating it being on.

½, 4	`Answer: 1`
	`transition(1,0,6,north,move_forward,3,north)`
	`dummy_transition(1,1,6,north,move_forward,3,north)`
	`transition(1,2,0,south,move_forward,3,south)`
	`dummy_transition(1,3,0,south,move_forward,3,south)`
	`Answer: 2`
	`transition(1,0,6,north,move_forward,3,north)`
	`dummy_transition(1,1,6,north,move_forward,3,north)`
	`transition(1,2,3,south,move_forward,6,south)`
	`dummy_transition(1,3,3,south,move_forward,6,south)`
	`Answer: 3`
	`transition(1,0,6,north,move_forward,3,north)`
	`dummy_transition(1,1,6,north,move_forward,3,north)`
		Is there the right answer among the ones given? Yes, 4th is the right answer: the train arrives at the destination at moment 4, 'lagging' in each cell for 2 seconds.
		
		The only thing, syntactically, that's missing to make it completely right, is 'dummy_transitions' being normal 'transitions' - for the present constraints to fire.

	transition(1,2,6,north,move_forward,3,north) dummy_transition(1,3,6,north,move_forward,3,north) **Answer: 4** **transition(1,0,6,north,move_forward,3,north)** **dummy_transition(1,1,6,north,move_forward,3,north)** **transition(1,2,3,north,move_forward,0,north)** **dummy_transition(1,3,3,north,move_forward,0,north)**	
¼, 8	I will copy only the right answer here this time: **Answer: 4** **transition(1,0,6,north,move_forward,3,north)** **dummy_transition(1,1,6,north,move_forward,3,north)** **dummy_transition(1,2,6,north,move_forward,3,north)** **dummy_transition(1,3,6,north,move_forward,3,north)** **transition(1,4,3,north,move_forward,0,north)** **dummy_transition(1,5,3,north,move_forward,0,north)** **dummy_transition(1,6,3,north,move_forward,0,north)** **dummy_transition(1,7,3,north,move_forward,0,north)**	Looks perfect: there are 3 dummy transitions dragging behind each real transition and copying them, with the only difference being the moments of time rolling forward.
⅛, 16	**Answer: 4** **transition(1,0,6,north,move_forward,3,north)** **dummy_transition(1,1,6,north,move_forward,3,north)** **dummy_transition(1,2,6,north,move_forward,3,north)** **dummy_transition(1,3,6,north,move_forward,3,north)** **dummy_transition(1,4,6,north,move_forward,3,north)** **dummy_transition(1,5,6,north,move_forward,3,north)** **dummy_transition(1,6,6,north,move_forward,3,north)** **dummy_transition(1,7,6,north,mo**	Again correct.

	ve_forward,3,north) transition(1,8,3,north,move_forward,0,north) dummy_transition(1,9,3,north,move_forward,0,north) dummy_transition(1,10,3,north,move_forward,0,north) dummy_transition(1,11,3,north,move_forward,0,north) dummy_transition(1,12,3,north,move_forward,0,north) dummy_transition(1,13,3,north,move_forward,0,north) dummy_transition(1,14,3,north,move_forward,0,north) dummy_transition(1,15,3,north,move_forward,0,north)	

Feels like I've implemented it - with one omission: constraints don't fire over 'dummy_transitions' because they don't see them. I need to convert both real and dummy transitions into one atom that my constraints would deal with. Since now we care more about a proof of concept than about computational efficiency, we can go for makeshift solutions that don't seem too elegant. Let's introduce an extra layer. **Doesn't help.** But at least I know where exactly the problem is: it is in the conflict between choice-rule created and normal rule created atoms of the same name. There is something that I don't understand about clingo's behaviour in this case.

Let's fixate this here to show to an expert if necessary:

This works perfectly:

```
dummy_transition(TRAIN, MOMENT, FROM_POS, IN_DIR, ACTION, TO_POS, OUT_DIR) :- intermediate_missing(TRAIN, MOMENT),
                                                                              action_transition(TRAIN, MOMENT-MARGIN, FROM_POS, IN_DIR, ACTION,
                                                                              margin(TRAIN, MARGIN).

transition(TRAIN, MOMENT, FROM_POS, IN_DIR, ACTION, TO_POS, OUT_DIR) :- action_transition(TRAIN, MOMENT, FROM_POS, IN_DIR, ACTION, TO_POS, OUT
```

And this become immediately unsatisfiable:

```
transition(TRAIN, MOMENT, FROM_POS, IN_DIR, ACTION, TO_POS, OUT_DIR) :- intermediate_missing(TRAIN, MOMENT),
                                                                         action_transition(TRAIN, MOMENT-MARGIN, FROM_POS, IN_DIR, ACTION,
                                                                         margin(TRAIN, MARGIN).

transition(TRAIN, MOMENT, FROM_POS, IN_DIR, ACTION, TO_POS, OUT_DIR) :- action_transition(TRAIN, MOMENT, FROM_POS, IN_DIR, ACTION, TO_POS, OUT
```

INTUITION: don't care about transitions vs dummy transitions - copy the constraints to cover the second, then convert them into 'action' too, so the interface is correct.

Let's see what it looks like if we only look at 'action' atoms: perfect, it works:

```
Answer: 1
action(1,1,move_forward) action(1,2,move_forward) action(1,3,move_forward) action(1,0,move_forward) action(1,5,move_forw
ard) action(1,6,move_forward) action(1,7,move_forward) action(1,4,move_forward)
Answer: 2
action(1,1,move_forward) action(1,2,move_forward) action(1,3,move_forward) action(1,0,move_forward) action(1,5,move_forw
ard) action(1,6,move_forward) action(1,7,move_forward) action(1,4,move_forward)
Answer: 3
action(1,1,move_forward) action(1,2,move_forward) action(1,3,move_forward) action(1,0,move_forward) action(1,5,move_forw
ard) action(1,6,move_forward) action(1,7,move_forward) action(1,4,move_forward)
Answer: 4
action(1,1,move_forward) action(1,2,move_forward) action(1,3,move_forward) action(1,0,move_forward) action(1,5,move_forw
ard) action(1,6,move_forward) action(1,7,move_forward) action(1,4,move_forward)
```

So, since the final output is actually correct anyway (and we only care about it in the end), I can just leave my 'dummy transitions' in place.

INTUITION: not all constraints are necessary for the dummy transitions - some are implicit for them.

INTUITION: it's also nice to introduce them (if it is possible) via disjunction: 'either a real or a dummy transition must not'.

LLM:

In Answer Set Programming (ASP), disjunctions are generally **not allowed** in constraints (rules with empty heads). Constraints are used to eliminate answer sets that violate certain conditions, and their syntax is:

This can help:

If the disjunction is complex, introduce an auxiliary predicate to represent it:

prolog Copy Download

```
disjunction :- a.
disjunction :- b.
:- disjunction.
```

So, this means in my case to introduce an atom 'reached', like in Jonas' encoding! Then, to build a constraint on it, not immediately on what I generate.

Introducing auxiliary atoms both for real and laggy transitions:

```
reached(TRAIN, MOMENT) :- transition(TRAIN,MOMENT,_,_,_,TO_POS,_),
                          end_flat(TRAIN, ARRIVAL_POS, ARRIVAL_TIME),
                          MOMENT == ARRIVAL_TIME-1,
                          TO_POS == ARRIVAL_POS.
```

```
reached(TRAIN, MOMENT+1) :- laggy_transition(TRAIN,MOMENT,_,_,_,TO_POS,_),
                            end_flat(TRAIN, ARRIVAL_POS, ARRIVAL_TIME),
                            MOMENT == ARRIVAL_TIME-1,
                            TO_POS == ARRIVAL_POS.
```

Good, now let's write the constraint itself - it will be very simple thanks to auxiliary atoms (this is an **additional benefit** of introducing auxiliary atoms - they simplify constraints):

```
:- end(TRAIN, (_, _), MOMENT), not reached(TRAIN, MOMENT).
```

This whole logic **works**, but it may be **too strict**, not letting the train arrive even before the stated arrival time (which is not a problem since trains **don't block each other** at the start / end).

This behaviour:

```
Answer: 1
reached(1,5) reached(1,6) reached(1,7) reached(1,8)
```

Is okay, since the train "covers" the end stretch in 4 moments of time. So I must make sure that only the latest moment of the laggy transition is considered actual arrival.

'reached' is not created if there is a laggy transition following on:

```
reached(TRAIN, MOMENT+1) :- transition(TRAIN,MOMENT,_,_,_,TO_POS,_),
                            not laggy_transition(TRAIN,MOMENT+MARGIN,_,_,_,TO_POS,_),
                            margin(TRAIN, MARGIN),
                            end_flat(TRAIN, ARRIVAL_POS, _),
                            TO_POS == ARRIVAL_POS.
```

'reached' is only created for the last laggy transition:

Fixed it back by introducing the hard constraint.

DIRECTIONS OF DEVELOPMENT:

1. Return all tracks and test more complex environments.
2. Return the opportunity to wait (now a fast train can't handle excessive time).
3. Run for several trains with different speeds.

4. Think about the concept of acceleration, i.e. simply *changing speed.*

INTUITION: there must be a constraint requiring enough time to arrive with the given speed. If there's not enough, there's no point in doing any other calculations.

IDEA: use Pygame to visualize movement of trains quicker without Flatland.

Q to an LLM: why when I change _ in my encoding it becomes unsatisfiable?

Suddenly, **a solid move forward today**.

___

27.06.25

Getting down to the tasks I set myself yesterday:

1. Return all tracks and test more complex environments.

So, first I'll simply copy them from the existing fully-functional encoding. As simple as that. Then run something more complex. I don't expect any problems from this step - it will only allow to create 'path' atoms for more complex tracks. The core speed logic - as changing frequency of train actions - must not change.

Let's just run with the previous input and see if it still works. **Works.** In terms of 'action' atoms, it looks perfect: the slower the speed, the more action atoms we get (the example is for speed 3):

```
action(1,1,move_forward) action(1,2,move_forward)
action(1,4,move_forward) action(1,5,move_forward)
action(1,0,move_forward) action(1,3,move_forward)
```

So, to cover the route of length 2, the train must make 6 actions at the speed of 3.

Let's now gradually try more complex tracks:

Environment	**Results & Commentary**
Γ-turn	Fails immediately - now every input requires the 'speed()' fact to be present. There are many atoms and rules bound to this input fact. Fixing. Speed 1: len Speed 1/2: many answer sets; why? H1: the constraints enforcing consistency are only fired for 'transition' atoms, not for 'laggy_transition' ones; let's fix this. The technology must be the same: via an auxiliary atom - 'consistent_movement ()' or simpler: 'inconsistent_movement'. Experimented by naively copying the 'transition' constraints. Let's take one answer and sort

it by ascending moments:

transition(1,0,6,north,move_forward,3,north),
laggy_transition(1,1,6,north,move_forward,3,north),
transition(1,2,3,south,move_forward,6,south),
laggy_transition(1,3,3,south,move_forward,6,south),
transition(1,4,3,south,move_forward,6,south),
laggy_transition(1,5,3,south,move_forward,6,south),
transition(1,6,1,east,move_forward,2,east),
laggy_transition(1,7,1,east,move_forward,2,east)

Yes, H1 is certainly **true:** it's about these consistency constraints. I need to expand them onto 'laggy_transitions'.

The inconsistencies are marked red now. I need to write a constraint that describes them:

a. The pattern seems to be unidirectional: a clenching of laggy_transition and a real transition is incorrect, not vice versa.

b. If there is another wrong pattern, we will see it through the remaining answer sets.

Now, let's focus on the constraint:

Constraint idea	**ASP phrasing**	**Code**
Firing output example: laggy_transition(1,1,6,north, move_forward,3,north), transition(1,2,3,south,move_ forward,6,south) There must be no answer set where there is a laggy_transition atom with certain output position and direction and an immediately following real transition with incoming position and direction different from those mentioned for the 'laggy' atom.	same	:- laggy_transition(TRAIN, MOMENT, _, _, _, OUT_POS, OUT_DIR), transition(TRAIN, MOMENT+1, IN_POS, IN_DIR, _, _, _), IN_POS != OUT_POS, IN_DIR != OUT_DIR.

Success. Much better now: 4 answer sets. Let's explore them to see what still creates this redundancy:

	Answer: 1
	transition(1,0,6,north,move_forward,3,north)
	laggy_transition(1,1,6,north,move_forward,**3,north**)
	transition(1,2,**0,north**,move_forward,1,east)
	laggy_transition(1,3,0,north,move_forward,1,east)
	transition(1,4,1,east,move_forward,2,east)
	laggy_transition(1,5,1,east,move_forward,**2,east**)
	transition(1,6,**1,east**,move_forward,2,east)
	laggy_transition(1,7,1,east,move_forward,2,east)
	
	Let's compare:
	
	`:- laggy_transition(TRAIN, MOMENT, _, _, _, OUT_POS, OUT_DIR),`
	`transition(TRAIN, MOMENT+1, IN_POS, IN_DIR, _, _, _),`
	`IN_POS != OUT_POS, IN_DIR != OUT_DIR.`
	
	Something feels off with the end of the track. So, again there are inconsistent transitions. What prevents them from being covered by the constraint?
	
	DISCOVERY: comma in a constraint means DISJUNCTION - so I needed two separate constraints.
	
	Success

Π-turn	**Success**
Bagel	I run with the default speed: the output is as before. I run with ½ of the speed: the output is twice longer, with the appropriate number of lagging transitions: `Answer: 1` `transition(1,0,5,east,move_forward,6,east) laggy_transition(1,1,5,east,move_forward,6,east) transition(1,2,6,east,turn_left,1,north) transition(1,4,1,north,move_forward,2,east) transition(1,6,2,east,move_forward,3,east) transition(1,8,3,east,move_forward,8,south) transition(1,10,8,south,move_forward,9,east) laggy_transition(1,3,6,east,turn_left,1,north) laggy_transition(1,5,1,north,move_forward,2,east) laggy_transition(1,7,2,east,move_forward,3,east) laggy_transition(1,9,3,east,move_forward,8,south) laggy_transition(1,11,8,south,move_forward,9,east)` `Answer: 2` `transition(1,0,5,east,move_forward,6,east) laggy_transition(1,1,5,east,move_forward,6,east) transition(1,2,6,east,turn_right,11,south) transition(1,4,11,south,move_forward,12,east) transition(1,6,12,east,move_forward,13,east) transition(1,8,13,east,move_forward,8,north) transition(1,10,8,north,move_forward,9,east) laggy_transition(1,3,6,east,turn_right,11,south) laggy_transition(1,5,11,south,move_forward,12,east) laggy_transition(1,7,12,east,move_forward,13,east) laggy_transition(1,9,13,east,move_forward,8,north) laggy_transition(1,11,8,north,move_forward,9,east)` There are two answers because there are two equally valued routes in the bagel. I set the speed to ¼ and the output is correct again:

	Answer: 1 transition(1,0,5,east,move_forward,6,east) laggy_transition(1,1,5,east,move_forward,6,east) laggy_transition(1,2,5,east,move_forward,6,east) laggy_transition(1,3,5,east,move_forward,6,east) transition(1,4,6,east,turn_left,1,north) transition(1,8,1,north,move_forward,2,east) transition(1,12,2,east,move_forward,3,east) transition(1,16,3,east,move_forward,8,south) transition(1,20,8,south,move_forward,9,east) laggy_transition(1,5,6,east,turn_left,1,north) laggy_transition(1,9,1,north,move_forward,2,east) laggy_transition(1,13,2,east,move_forward,3,east) laggy_transition(1,17,3,east,move_forward,8,south) laggy_transition(1,21,8,south,move_forward,9,east) laggy_transition(1,6,6,east,turn_left,1,north) laggy_transition(1,10,1,north,move_forward,2,east) laggy_transition(1,14,2,east,move_forward,3,east) laggy_transition(1,18,3,east,move_forward,8,south) laggy_transition(1,22,8,south,move_forward,9,east) laggy_transition(1,7,6,east,turn_left,1,north) laggy_transition(1,11,1,north,move_forward,2,east) laggy_transition(1,15,2,east,move_forward,3,east) laggy_transition(1,19,3,east,move_forward,8,south) laggy_transition(1,23,8,south,move_forward,9,east) Answer: 2 transition(1,0,5,east,move_forward,6,east) laggy_transition(1,1,5,east,move_forward,6,east) laggy_transition(1,2,5,east,move_forward,6,east) laggy_transition(1,3,5,east,move_forward,6,east) transition(1,4,6,east,turn_right,11,south) transition(1,8,11,south,move_forward,12,east) transition(1,12,12,east,move_forward,13,east) transition(1,16,13,east,move_forward,8,north) transition(1,20,8,north,move_forward,9,east) laggy_transition(1,5,6,east,turn_right,11,south) laggy_transition(1,9,11,south,move_forward,12,east) laggy_transition(1,13,12,east,move_forward,13,east) laggy_transition(1,17,13,east,move_forward,8,north) laggy_transition(1,21,8,north,move_forward,9,east) laggy_transition(1,6,6,east,turn_right,11,south) laggy_transition(1,10,11,south,move_forward,12,east) laggy_transition(1,14,12,east,move_forward,13,east) laggy_transition(1,18,13,east,move_forward,8,north) laggy_transition(1,22,8,north,move_forward,9,east) laggy_transition(1,7,6,east,turn_right,11,south) laggy_transition(1,11,11,south,move_forward,12,east) laggy_transition(1,15,12,east,move_forward,13,east) laggy_transition(1,19,13,east,move_forward,8,north) laggy_transition(1,23,8,north,move_forward,9,east) **Success**
Many turns	Works for speed of 1 perfectly, as before: transition(1,0,1,south,move_forward,5,south) transition(1,2,9,south,move_forward,13,south) transition(1,1,5,south,move_forward,9,south) Switching to ½: Answer: 1 transition(1,0,1,south,move_forward,5,south) transition(1,4,9,south,move_forward,13,south) laggy_transition(1,1,1,south,move_forward,5,south) laggy_transition(1,5,9,south,move_forward,13,south) transition(1,2,5,south,move_forward,9,south) laggy_transition(1,3,5,south,move_forward,9,south) To ¼: Answer: 1 transition(1,0,1,south,move_forward,5,south) transition(1,8,9,south,move_forward,13,south) laggy_transition(1,1,1,south,move_forward,5,south) laggy_transition(1,9,9,south,move_forward,13,south) laggy_transition(1,2,1,south,move_forward,5,south) laggy_transition(1,10,9,south,move_forward,13,south) laggy_transition(1,3,1,south,move_forward,5,south) laggy_transition(1,11,9,south,move_forward,13,south) transition(1,4,5,south,move_forward,9,south) laggy_transition(1,5,5,south,move_forward,9,south) laggy_transition(1,6,5,south,move_forward,9,south) laggy_transition(1,7,5,south,move_forward,9,south) To ⅛: Answer: 1 transition(1,0,1,south,move_forward,5,south) transition(1,16,9,south,move_forward,13,south) laggy_transition(1,1,1,south,move_forward,5,south) laggy_transition(1,2,1,south,move_forward,5,south) laggy_transition(1,3,1,south,move_forward,5,south) laggy_transition(1,4,1,south,move_forward,5,south) laggy_transition(1,5,1,south,move_forward,5,south) laggy_transition(1,6,1,south,move_forward,5,south) laggy_transition(1,7,1,south,move_forward,5,south) laggy_transition(1,17,9,south,move_forward,13,south) laggy_transition(1,18,9,south,move_forward,13,south) laggy_transition(1,19,9,south,move_forward,13,south) laggy_transition(1,20,9,south,move_forward,13,south) laggy_transition(1,21,9,south,move_forward,13,south) laggy_transition(1,22,9,south,move_forward,13,south) laggy_transition(1,23,9,south,move_forward,13,south) transition(1,8,5,south,move_forward,9,south) laggy_transition(1,9,5,south,move_forward,9,south) laggy_transition(1,10,5,south,move_forward,9,south) laggy_transition(1,11,5,south,move_forward,9,south) laggy_transition(1,12,5,south,move_forward,9,south) laggy_transition(1,13,5,south,move_forward,9,south) laggy_transition(1,14,5,south,move_forward,9,south) laggy_transition(1,15,5,south,move_forward,9,south) **Success**
Switch trap	Speed 1: transition(1,0,3,east,move_forward,4,east) transition(1,2,7,south,move_forward,8,east) transition(1,1,4,east,turn_right,7,south)

	Speed ½:
	
	`transition(1,0,3,east,move_forward,4,east) transition(1,4,7,south,move_forward,8,east) laggy_transition(1,1,3,east,move_forward,4,east) laggy_transition(1,5,7,south,move_forward,8,east) transition(1,2,4,east,turn_right,7,south) laggy_transition(1,3,4,east,turn_right,7,south)`
	
	Speed ¼:
	
	`transition(1,0,3,east,move_forward,4,east) transition(1,8,7,south,move_forward,8,east) laggy_transition(1,1,3,east,move_forward,4,east) laggy_transition(1,9,7,south,move_forward,8,east) laggy_transition(1,2,3,east,move_forward,4,east) laggy_transition(1,10,7,south,move_forward,8,east) laggy_transition(1,3,3,east,move_forward,4,east) laggy_transition(1,11,7,south,move_forward,8,east) transition(1,4,4,east,turn_right,7,south) laggy_transition(1,5,4,east,turn_right,7,south) laggy_transition(1,6,4,east,turn_right,7,south) laggy_transition(1,7,4,east,turn_right,7,south)`
	
	Success

28.06.25

Now, I'll just keep testing the existing speed implementation on more environments, looking for bugs and fixing them should they occur (see table above).

Now testing for multiply trains (this will reveal the true value of my 'speed' approach - if it works for 2 trains of different speeds, it's **robust and universal**):

Environment	**Results & Commentary**
diamond	Testing with default speed of 1: **Fails.** Why? Let's switch off one of the trains. Works. Another: **fails.** Two reasons: a. Mistakenly set the speed for the 1st instead of the 2nd (this goes into my 'mistake statistics' - most reasons why a big encoding fails are **lowest level bugs**) b. There is no **waiting**! Without waiting, running two trains is only possible when they luckily miss each other. Let's just copy the old waiting logic here from the full-functional motion_5.lp: **Done.** IMPLICATION: waiting will also immediately take 'speed' amount of moments - because *any* action (including waiting) just takes longer for a slower train (I'm not quite sure it's a problem at this stage; later, it might be proved too inconsistent with realistic movement) I decided to add incrementally and just allowed the trains, if necessary, wait at the departure point. It worked for speed 1:

```
transition(1,3,4,east,move_forward,5,east) transition(2,1,4,south,move_forward,7,south) transition(1,0,3,east,wait,3,east) transition(1,1,3,east,wait,3,east) transition(1,2,3,east,move_forward,4,east) transition(2,0,1,south,move_forward,4,south)
```

One of the trains simply waits at the start while the other passes.

Speed ½ for both trains:

```
transition(1,4,4,east,move_forward,5,east) transition(2,2,4,south,move_forward,7,south) laggy_transition(2,3,4,south,move_forward,7,south) transition(1,0,3,east,wait,3,east) transition(1,2,3,east,move_forward,4,east) transition(2,0,1,south,move_forward,4,south) laggy_transition(1,1,3,east,wait,3,east) laggy_transition(1,3,3,east,move_forward,4,east) laggy_transition(2,1,1,south,move_forward,4,south)
```

Laggy transitions appeared for both trains. All looks correct.

Speed ½ for the first train: **fails**

Exploring: let's run just one of the trains separately with speed 1/2.: **works.**

Made it work: the lag was most likely in the train of speed 1 - I set the arrival time to be 4, while it only needed to cover 2 cells. My arrival time constraint now is **overly tight for simplicity and must be loosened for production.**

One of the correctness criteria: there must be **no 'laggy transitions' for a train with speed of 1 (train 2)**:

transition(1,4,4,east,move_forward,5,east) transition(2,1,4,south,move_forward,7,south) laggy_transition(1,5,4,east,move_forward,5,east) transition(1,0,3,east,wait,3,east) transition(1,2,3,east,move_forward,4,east) transition(2,0,1,south,move_forward,4,south) laggy_transition(1,1,3,east,wait,3,east) laggy_transition(1,3,3,east,move_forward,4,east)

Yes, all the laggy transitions are for the lower speed train, so may conclude that **at least partially the implementation of speed is correct**.

Let's now make both trains slower than 1: ½ and ½ (one also gets a leeway for waiting):

```
Answer: 1
transition(1,4,4,east,move_forward,5,east) transition(2,2,4,south,move_forward,7,south) laggy_transition(1,5,4,east,move_forward,5,east) laggy_transition(2,3,4,south,move_forward,7,south) transition(1,0,3,east,wait,3,east) transition(1,2,3,east,move_forward,4,east) transition(2,0,1,south,move_forward,4,south) laggy_transition(1,1,3,east,wait,3,east) laggy_transition(1,3,3,east,move_forward,4,east) laggy_transition(2,1,1,south,move_forward,4,south)
```

Partial correctness criteria: both trains must get some laggy transitions. Checking: **satisfied.** Conclusion: my speed implementation is at least partially correct.

Let's approach edge cases and set speeds to be radically different (domain example: ICE vs a heavy loaded freight train):

First train: speed ¼, leeway on arrival
Second train: speed 1, no leeway

Answer:

	Answer: 1 transition(1,16,4,east,move_forward,5,east) transition(2,1,4,south,move_forward,7,south) laggy_transition(1,17,4,east,move_forward,5,east) transition(1,0,3,east,wait,3,east) transition(1,4,3,east,wait,3,east) transition(1,8,3,east,wait,3,east) transition(1,12,3,east,move_forward,4,east) transition(2,0,1,south,move_forward,4,south) laggy_transition(1,1,3,east,wait,3,east) laggy_transition(1,5,3,east,wait,3,east) laggy_transition(1,9,3,east,wait,3,east) laggy_transition(1,13,3,east,move_forward,4,east) laggy_transition(1,2,3,east,wait,3,east) laggy_transition(1,6,3,east,wait,3,east) laggy_transition(1,10,3,east,wait,3,east) laggy_transition(1,14,3,east,move_forward,4,east) laggy_transition(1,3,3,east,wait,3,east) laggy_transition(1,7,3,east,wait,3,east) laggy_transition(1,11,3,east,wait,3,east) laggy_transition(1,15,3,east,move_forward,4,east) **Partial correctness criterion:** no laggy transitions for 'ICE', 3 laggy transitions for each 'real' transition for the freight. Checking: apparently **correct**. **Success.**
single switch	Testing with default speeds of 1 for both trains: transition(2,4,16,east,move_forward,17,east) transition(1,0,0,east,move_forward,1,east) transition(1,1,1,east,move_forward,2,east) transition(1,2,2,east,move_forward,8,south) transition(1,3,8,south,move_forward,14,south) transition(1,4,14,south,move_forward,13,west) transition(1,5,13,west,move_forward,12,west) transition(2,0,12,east,move_forward,13,east) transition(2,1,13,east,move_forward,14,east) transition(2,2,14,east,move_forward,15,east) transition(2,3,15,east,move_forward,16,east) No laggy transitions - **correct**. No waiting either - the trains 'get lucky' here. **Correct.** Slowing both trains down to ½: Answer: 1 transition(2,8,16,east,move_forward,17,east) laggy_transition(2,9,16,east,move_forward,17,east) transition(1,0,0,east,move_forward,1,east) transition(1,2,1,east,move_forward,2,east) transition(1,4,2,east,move_forward,8,south) transition(1,6,8,south,move_forward,14,south) transition(1,8,14,south,move_forward,13,west) transition(1,10,13,west,move_forward,12,west) transition(2,0,12,east,move_forward,13,east) transition(2,2,13,east,move_forward,14,east) transition(2,4,14,east,move_forward,15,east) transition(2,6,15,east,move_forward,16,east) laggy_transition(1,1,0,east,move_forward,1,east) laggy_transition(1,3,1,east,move_forward,2,east) laggy_transition(1,5,2,east,move_forward,8,south) laggy_transition(1,7,8,south,move_forward,14,south) laggy_transition(1,9,14,south,move_forward,13,west) laggy_transition(1,11,13,west,move_forward,12,west) laggy_transition(2,1,12,east,move_forward,13,east) laggy_transition(2,3,13,east,move_forward,14,east) laggy_transition(2,5,14,east,move_forward,15,east) laggy_transition(2,7,15,east,move_forward,16,east) Appears **correct**. Let's look at 'action' representation - it's important for the graphical interface: action(2,9,move_forward) action(2,8,move_forward) action(1,1,move_forward) action(1,3,move_forward) action(1,5,move_forward) action(1,7,move_forward) action(1,9,move_forward) action(1,11,move_forward) action(2,1,move_forward) action(2,3,move_forward) action(2,5,move_forward) action(2,7,move_forward) action(1,0,move_forward) action(1,2,move_forward) action(1,4,move_forward) action(1,6,move_forward) action(1,8,move_forward) action(1,10,move_forward) action(2,0,move_forward) action(2,2,move_forward) action(2,4,move_forward) action(2,6,move_forward) Appears **correct**. Default speed for one train, ½ for another: action(2,9,move_forward) action(2,8,move_forward) action(2,1,move_forward) action(2,3,move_forward) action(2,5,move_forward) action(2,7,move_forward) action(1,0,move_forward) action(1,1,move_forward) action(1,2,move_forward) action(1,3,move_forward) action(1,4,move_forward) action(1,5,move_forward) action(2,0,move_forward) action(2,2,move_forward) action(2,4,move_forward) action(2,6,move_forward) The answer set in 'action' atoms is shorter. Let's see the transitions: transition(2,8,16,east,move_forward,17,east) laggy_transition(2,9,16,east,move_forward,17,east) transition(1,0,0,east,move_forward,1,east) transition(1,1,1,east,move_forward,2,east) transition(1,2,2,east,move_forward,8,south) transition(1,3,8,south,move_forward,14,south) transition(1,4,14,south,move_forward,13,west) transition(1,5,13,west,move_forward,12,west) transition(2,0,12,east,move_forward,13,east) transition(2,2,13,east,move_forward,14,east) transition(2,4,14,east,move_forward,15,east) transition(2,6,15,east,move_forward,16,east) laggy_transition(2,1,12,east,move_forward,13,east) laggy_transition(2,3,13,east,move_forward,14,east) laggy_transition(2,5,14,east,move_forward,15,east) laggy_transition(2,7,15,east,move_forward,16,east)

bowel	Testing ‘ICE vs freight’: Answer: 1 transition(2,18,16,east,move_forward,17,east) laggy_transition(2,19,16,east,move_forward,17,east) transition(1,0,0,east,move_forward,1,east) transition(1,1,1,east,move_forward,2,east) transition(1,2,2,east,move_forward,8,south) transition(1,3,8,south,move_forward,14,south) transition(1,4,14,south,move_forward,13,west) transition(1,5,13,west,move_forward,12,west) transition(2,0,12,east,wait,12,east) transition(2,3,12,east,wait,12,east) transition(2,6,12,east,move_forward,13,east) transition(2,9,13,east,move_forward,14,east) transition(2,12,14,east,move_forward,15,east) transition(2,15,15,east,move_forward,16,east) laggy_transition(2,1,12,east,wait,12,east) laggy_transition(2,4,12,east,wait,12,east) laggy_transition(2,7,12,east,move_forward,13,east) laggy_transition(2,10,13,east,move_forward,14,east) laggy_transition(2,13,14,east,move_forward,15,east) laggy_transition(2,16,15,east,move_forward,16,east) laggy_transition(2,2,12,east,wait,12,east) laggy_transition(2,5,12,east,wait,12,east) laggy_transition(2,8,12,east,move_forward,13,east) laggy_transition(2,11,13,east,move_forward,14,east) laggy_transition(2,14,14,east,move_forward,15,east) laggy_transition(2,17,15,east,move_forward,16,east) Appears **correct.** Let’s finally turn on the opportunity to wait **before a junction** - not only at the start: **Done.** Retesting: this may cause unexpected results, as usual with asp: 3 answer sets instead of 1 (EXPLANATION: two more places to wait). Tightening the time: the tightest time is 16 for the freight, but there are still 2 answer sets. It works, so it’s not a problem - checking every atom is too tedious. **Success.**
caduceus	Running with the default speed of 1: **works:** transition(2,11,17,east,move_forward,20,south) transition(1,0,1,south,move_forward,4,south) transition(1,1,4,south,move_forward,7,south) transition(1,2,7,south,move_forward,10,south) transition(1,3,10,south,move_forward,13,south) transition(1,4,13,south,move_forward,16,south) transition(1,5,16,south,move_forward,19,south) transition(2,0,0,east,move_forward,3,south) transition(2,1,3,south,move_forward,4,east) transition(2,2,4,east,move_forward,5,east) transition(2,3,5,east,move_forward,8,south) transition(2,4,8,south,move_forward,11,south) transition(2,5,11,south,move_forward,10,west) transition(2,6,10,west,move_forward,9,west) transition(2,7,9,west,move_forward,12,south) transition(2,8,12,south,move_forward,15,south) transition(2,9,15,south,move_forward,16,east) transition(2,10,16,east,move_forward,17,east) Running with speed of ½ for both trains: transition(2,22,17,east,move_forward,20,south) laggy_transition(2,23,17,east,move_forward,20,south) transition(1,0,1,south,move_forward,4,south) transition(1,2,4,south,move_forward,7,south) transition(1,4,7,south,move_forward,10,south) transition(1,6,10,south,move_forward,13,south) transition(1,8,13,south,move_forward,16,south) transition(1,10,16,south,move_forward,19,south) transition(2,0,0,east,move_forward,3,south) transition(2,2,3,south,move_forward,4,east) transition(2,4,4,east,move_forward,5,east) transition(2,6,5,east,move_forward,8,south) transition(2,8,8,south,move_forward,11,south) transition(2,10,11,south,move_forward,10,west) transition(2,12,10,west,move_forward,9,west) transition(2,14,9,west,move_forward,12,south) transition(2,16,12,south,move_forward,15,south) transition(2,18,15,south,move_forward,16,east) transition(2,20,16,east,move_forward,17,east) laggy_transition(1,1,1,south,move_forward,4,south) laggy_transition(1,3,4,south,move_forward,7,south) laggy_transition(1,5,7,south,move_forward,10,south) laggy_transition(1,7,10,south,move_forward,13,south) laggy_transition(1,9,13,south,move_forward,16,south) laggy_transition(1,11,16,south,move_forward,19,south) laggy_transition(2,1,0,east,move_forward,3,south) laggy_transition(2,3,3,south,move_forward,4,east) laggy_transition(2,5,4,east,move_forward,5,east) laggy_transition(2,7,5,east,move_forward,8,south) laggy_transition(2,9,8,south,move_forward,11,south) laggy_transition(2,11,11,south,move_forward,10,west) laggy_transition(2,13,10,west,move_forward,9,west) laggy_transition(2,15,9,west,move_forward,12,south) laggy_transition(2,17,12,south,move_forward,15,south) laggy_transition(2,19,15,south,move_forward,16,east) laggy_transition(2,21,16,east,move_forward,17,east) Single answer. Appears **correct**. ‘ICE’ vs ‘freight’ (freight also takes a longer path, with ICE running on a separate straight track, which appears realistic):

	Answer: 1 transition(2,33,17,east,move_forward,20,south) laggy_transition(2,34,17,east,move_forward,20,south) laggy_transition(2,35,17,east,move_forward,20,south) transition(1,0,1,south,move_forward,4,south) transition(1,1,4,south,move_forward,7,south) transition(1,2,7,south,move_forward,10,south) transition(1,3,10,south,move_forward,13,south) transition(1,4,13,south,move_forward,16,south) transition(1,5,16,south,move_forward,19,south) transition(2,0,0,east,move_forward,3,south) transition(2,3,3,south,move_forward,4,east) transition(2,6,4,east,move_forward,5,east) transition(2,9,5,east,move_forward,8,south) transition(2,12,8,south,move_forward,11,south) transition(2,15,11,south,move_forward,10,west) transition(2,18,10,west,move_forward,9,west) transition(2,21,9,west,move_forward,12,south) transition(2,24,12,south,move_forward,15,south) transition(2,27,15,south,move_forward,16,east) transition(2,30,16,east,move_forward,17,east) laggy_transition(2,1,0,east,move_forward,3,south) laggy_transition(2,4,3,south,move_forward,4,east) laggy_transition(2,7,4,east,move_forward,5,east) laggy_transition(2,10,5,east,move_forward,8,south) laggy_transition(2,13,8,south,move_forward,11,south) laggy_transition(2,16,11,south,move_forward,10,west) laggy_transition(2,19,10,west,move_forward,9,west) laggy_transition(2,22,9,west,move_forward,12,south) laggy_transition(2,25,12,south,move_forward,15,south) laggy_transition(2,28,15,south,move_forward,16,east) laggy_transition(2,31,16,east,move_forward,17,east) laggy_transition(2,2,0,east,move_forward,3,south) laggy_transition(2,5,3,south,move_forward,4,east) laggy_transition(2,8,4,east,move_forward,5,east) laggy_transition(2,11,5,east,move_forward,8,south) laggy_transition(2,14,8,south,move_forward,11,south) laggy_transition(2,17,11,south,move_forward,10,west) laggy_transition(2,20,10,west,move_forward,9,west) laggy_transition(2,23,9,west,move_forward,12,south) laggy_transition(2,26,12,south,move_forward,15,south) laggy_transition(2,29,15,south,move_forward,16,east) laggy_transition(2,32,16,east,move_forward,17,east) Solves fast. Appears **correct**. I ascribe speed of solving to the fact that 'laggy transitions' are just derived with a normal rule from the real generator-made transitions, giving **little-to-none computational overhead**. **Success**
ten by ten	Wow, there are 4 trains already! Let's run with the default speed of 1 just to check if we haven't broken some core logic accidentally: **Fails** due to a dumb mistake - IDs start with 0 here, not with 1 ('mistake statistics': **always look for the dumbest reason when a complex encoding doesn't work)**. 22 answer sets. Why not? Let's also turn on the optimizer imperative and see what it does to them all: Optimum is found, and very quickly: transition(0,0,38,north,move_forward,28,north) transition(0,1,28,north,move_forward,18,north) transition(0,2,18,north,move_forward,17,west) transition(0,3,17,west,move_forward,16,west) transition(0,4,16,west,move_forward,15,west) transition(0,5,15,west,move_forward,25,south) transition(0,6,25,south,move_forward,35,south) transition(0,7,35,south,turn_right,34,west) transition(0,8,34,west,move_forward,33,west) transition(0,9,33,west,move_forward,43,south) transition(0,10,43,south,move_forward,53,south) transition(0,11,53,south,move_forward,63,south) transition(1,0,86,west,move_forward,85,west) transition(1,1,85,west,move_forward,75,north) transition(1,2,75,north,turn_left,74,west) transition(1,3,74,west,move_forward,73,west) transition(1,4,73,west,move_forward,83,south) transition(1,5,83,south,move_forward,82,west) transition(1,6,82,west,move_forward,81,west) transition(1,7,81,west,move_forward,71,north) transition(1,8,71,north,move_forward,61,north) transition(1,9,61,north,move_forward,62,east) transition(1,10,62,east,move_forward,52,north) transition(1,11,52,north,move_forward,53,east) transition(1,12,53,east,move_forward,43,north) transition(1,13,43,north,turn_left,42,west) transition(1,14,42,west,move_forward,41,west) transition(1,15,41,west,move_forward,31,north) transition(1,16,31,north,move_forward,21,north) transition(2,0,58,north,wait,58,north) transition(2,1,58,north,move_forward,57,west) transition(2,2,57,west,move_forward,56,west) transition(2,3,56,west,wait,56,west) transition(2,4,56,west,wait,56,west) transition(2,5,56,west,move_forward,46,north) transition(2,6,46,north,turn_left,45,west) transition(2,7,45,west,move_forward,35,north) transition(2,8,35,north,move_forward,25,north) transition(2,9,25,north,move_forward,15,north) transition(2,10,15,north,move_forward,16,east) transition(3,0,65,north,move_forward,66,east) transition(3,1,66,east,move_forward,56,north) transition(3,2,56,north,move_forward,46,north) transition(3,3,46,north,turn_right,47,east) Optimization: 44 OPTIMUM FOUND Looks **correct**. Let's now carefully vary speeds. First, let's just make one train lag: for example, the one with the shortest route.

```
train(3).
start(3, (6, 5), 0, north).
end(3, (4, 7), 4).
speed(3,1).
```

This guy has just 4 cells to cover. Let's make it a heavily loaded local freight train. 'Shifter locomotive' is a proper term for this, here's one:

Let its speed be thrice slower than the default one: ⅓.

```
transition(0,0,38,north,move_forward,28,north) transition(0,1,28,north,move_forward,18,north) transition(0,2,18,north,move_forward,17,west) transition(0,3,17,west,move_forward,16,west) transition(0,4,16,west,move_forward,15,west) transition(0,5,15,west,move_forward,25,south) transition(0,6,25,south,move_forward,35,south) transition(0,7,35,south,turn_right,34,west) transition(0,8,34,west,move_forward,33,west) transition(0,9,33,west,move_forward,43,south) transition(0,10,43,south,move_forward,53,south) transition(0,11,53,south,move_forward,63,south) transition(1,0,86,west,move_forward,85,west) transition(1,1,85,west,move_forward,75,north) transition(1,2,75,north,turn_left,74,west) transition(1,3,74,west,move_forward,73,west) transition(1,4,73,west,move_forward,83,south) transition(1,5,83,south,move_forward,82,west) transition(1,6,82,west,move_forward,81,west) transition(1,7,81,west,move_forward,71,north) transition(1,8,71,north,move_forward,61,north) transition(1,9,61,north,move_forward,62,east) transition(1,10,62,east,move_forward,52,north) transition(1,11,52,north,move_forward,53,east) transition(1,12,53,east,move_forward,43,north) transition(1,13,43,north,turn_left,42,west) transition(1,14,42,west,move_forward,41,west) transition(1,15,41,west,move_forward,31,north) transition(1,16,31,north,move_forward,21,north) transition(2,0,58,north,wait,58,north) transition(2,1,58,north,wait,58,north) transition(2,2,58,north,move_forward,57,west) transition(2,3,57,west,wait,57,west) transition(2,4,57,west,move_forward,56,west) transition(2,5,56,west,move_forward,46,north) transition(2,6,46,north,turn_left,45,west) transition(2,7,45,west,move_forward,35,north) transition(2,8,35,north,move_forward,25,north) transition(2,9,25,north,move_forward,15,north) transition(2,10,15,north,move_forward,16,east) transition(3,0,65,north,move_forward,66,east) transition(3,3,66,east,move_forward,56,north) transition(3,6,56,north,move_forward,46,north) transition(3,9,46,north,turn_right,47,east) laggy_transition(3,1,65,north,move_forward,66,east) laggy_transition(3,4,66,east,move_forward,56,north) laggy_transition(3,7,56,north,move_forward,46,north) laggy_transition(3,10,46,north,turn_right,47,east) laggy_transition(3,2,65,north,move_forward,66,east) laggy_transition(3,5,66,east,move_forward,56,north) laggy_transition(3,8,56,north,move_forward,46,north) laggy_transition(3,11,46,north,turn_right,47,east)
```

Appears **correct**.

Let's make two trains 'normal' (½ speed) and 1 'ICE' (longest haul):

```
train(0). % Local
start(0, (3, 8), 0, north).
end(0, (6, 3), 24).
speed(0,2).

train(1). % ICE
start(1, (8, 6), 0, west).
end(1, (2, 1), 17).
speed(1,1).

train(2). % Local
start(2, (5, 8), 1, north).
end(2, (1, 6), 22).
speed(2,2).

train(3). % Shifter
start(3, (6, 5), 0, north).
end(3, (4, 7), 12).
speed(3,3).
```

The input now looks like this.

IDEA: from the point of usability / user intuition, it would make sense to bind prioritization to the type of train instead of ID as an extra layer.

	The encoding really flies here: `Answer: 1` `transition(0,0,38,north,move_forward,28,north) transition(0,2,28,north,move_forward,18,north) transition(0,4,18,north,move_forward,17,west) transition(0,6,17,west,move_forward,16,west) transition(0,8,16,west,move_forward,15,west) transition(0,10,15,west,move_forward,25,south) transition(0,12,25,south,move_forward,35,south) transition(0,14,35,south,turn_right,34,west) transition(0,16,34,west,move_forward,33,west) transition(0,18,33,west,move_forward,43,south) transition(0,20,43,south,move_forward,53,south) transition(0,22,53,south,move_forward,63,south) transition(1,0,86,west,move_forward,85,west) transition(1,1,85,west,move_forward,75,north) transition(1,2,75,north,turn_left,74,west) transition(1,3,74,west,move_forward,73,west) transition(1,4,73,west,move_forward,83,south) transition(1,5,83,south,move_forward,82,west) transition(1,6,82,west,move_forward,81,west) transition(1,7,81,west,move_forward,71,north) transition(1,8,71,north,move_forward,61,north) transition(1,9,61,north,move_forward,62,east) transition(1,10,62,east,move_forward,52,north) transition(1,11,52,north,move_forward,53,east) transition(1,12,53,east,move_forward,43,north) transition(1,13,43,north,turn_left,42,west) transition(1,14,42,west,move_forward,41,west) transition(1,15,41,west,move_forward,31,north) transition(1,16,31,north,move_forward,21,north) transition(2,0,46,north,turn_left,45,west) transition(2,2,45,west,wait,45,west) transition(2,4,45,west,wait,45,west) transition(2,6,45,west,wait,45,west) transition(2,8,45,west,wait,45,west) transition(2,10,45,west,wait,45,west) transition(2,12,45,west,wait,45,west) transition(2,14,45,west,move_forward,35,north) transition(2,16,35,north,move_forward,25,north) transition(2,18,25,north,move_forward,15,north) transition(2,20,15,north,move_forward,16,east) transition(3,0,65,north,move_forward,66,east) transition(3,3,66,east,move_forward,56,north) transition(3,6,56,north,move_forward,46,north) transition(3,9,46,north,turn_right,47,east) laggy_transition(0,1,38,north,move_forward,28,north) laggy_transition(0,3,28,north,move_forward,18,north) laggy_transition(0,5,18,north,move_forward,17,west) laggy_transition(0,7,17,west,move_forward,16,west) laggy_transition(0,9,16,west,move_forward,15,west) laggy_transition(0,11,15,west,move_forward,25,south) laggy_transition(0,13,25,south,move_forward,35,south) laggy_transition(0,15,35,south,turn_right,34,west) laggy_transition(0,17,34,west,move_forward,33,west) laggy_transition(0,19,33,west,move_forward,43,south) laggy_transition(0,21,43,south,move_forward,53,south) laggy_transition(0,23,53,south,move_forward,63,south) laggy_transition(2,1,46,north,turn_left,45,west) laggy_transition(2,3,45,west,wait,45,west) laggy_transition(2,5,45,west,wait,45,west) laggy_transition(2,7,45,west,wait,45,west) laggy_transition(2,9,45,west,wait,45,west) laggy_transition(2,11,45,west,wait,45,west) laggy_transition(2,13,45,west,wait,45,west) laggy_transition(2,15,45,west,move_forward,35,north) laggy_transition(2,17,35,north,move_forward,25,north) laggy_transition(2,19,25,north,move_forward,15,north) laggy_transition(2,21,15,north,move_forward,16,east) laggy_transition(3,1,65,north,move_forward,66,east) laggy_transition(3,4,66,east,move_forward,56,north) laggy_transition(3,7,56,north,move_forward,46,north) laggy_transition(3,10,46,north,turn_right,47,east) laggy_transition(3,2,65,north,move_forward,66,east) laggy_transition(3,5,66,east,move_forward,56,north) laggy_transition(3,8,56,north,move_forward,46,north) laggy_transition(3,11,46,north,turn_right,47,east)` `Optimization: 75` Optimum found for these different speed trains **in just 0.13 seconds**. And finding optimum, according to Kep, **is not even mandatory**. IDEA: settle for this speed range and increments: 1 (ICE), 1/2 (local), ⅓ (freight / shifter). There's no need to squeeze more out of this encoding. Everything else would be 'edge cases'. **Success.**

Large 1	Running with default speeds for both trains.
	
	Solves quickly with optimization:
	`Answer: 1`
	`transition(0,0,185,south,move_forward,215,south) transition(0,1,215,south,move_forward,216,east) transition(0,2,216,east,wait,216,east) transition(0,3,216,east,wait,216,east) transition(0,4,216,east,wait,216,east) transition(0,5,216,east,move_forward,246,south) transition(0,6,246,south,wait,246,south) transition(0,7,246,south,turn_right,247,east) transition(0,8,247,east,move_forward,277,south) transition(0,9,277,south,wait,277,south) transition(0,10,277,south,wait,277,south) transition(0,11,277,south,wait,277,south) transition(0,12,277,south,move_forward,307,south) transition(0,13,307,south,wait,307,south) transition(0,14,307,south,wait,307,south) transition(0,15,307,south,wait,307,south) transition(0,16,307,south,wait,307,south) transition(0,17,307,south,wait,307,south) transition(0,18,307,south,move_forward,337,south) transition(0,19,337,south,move_forward,367,south) transition(0,20,367,south,move_forward,397,south) transition(0,21,397,south,move_forward,427,south) transition(0,22,427,south,move_forward,457,south) transition(0,23,457,south,move_forward,487,south) transition(0,24,487,south,move_forward,517,south) transition(0,25,517,south,move_forward,547,south) transition(0,26,547,south,move_forward,577,south) transition(0,27,577,south,move_forward,607,south) transition(0,28,607,south,move_forward,637,south) transition(0,29,637,south,move_forward,667,south) transition(0,30,667,south,move_forward,697,south) transition(0,31,697,south,move_forward,727,south) transition(1,0,726,north,wait,726,north) transition(1,1,726,north,wait,726,north) transition(1,2,726,north,wait,726,north) transition(1,3,726,north,move_forward,696,north) transition(1,4,696,north,move_forward,666,north) transition(1,5,666,north,turn_right,667,east) transition(1,6,667,east,move_forward,637,north) transition(1,7,637,north,turn_left,636,west) transition(1,8,636,west,move_forward,606,north) transition(1,9,606,north,move_forward,576,north) transition(1,10,576,north,move_forward,546,north) transition(1,11,546,north,move_forward,516,north) transition(1,12,516,north,move_forward,486,north) transition(1,13,486,north,move_forward,456,north) transition(1,14,456,north,move_forward,426,north) transition(1,15,426,north,move_forward,396,north) transition(1,16,396,north,wait,396,north) transition(1,17,396,north,move_forward,366,north) transition(1,18,366,north,move_forward,336,north) transition(1,19,336,north,move_forward,306,north) transition(1,20,306,north,wait,306,north) transition(1,21,306,north,wait,306,north) transition(1,22,306,north,wait,306,north) transition(1,23,306,north,wait,306,north) transition(1,24,306,north,move_forward,276,north) transition(1,25,276,north,turn_right,277,east) transition(1,26,277,east,move_forward,247,north) transition(1,27,247,north,wait,247,north) transition(1,28,247,north,wait,247,north) transition(1,29,247,north,wait,247,north) transition(1,30,247,north,wait,247,north) transition(1,31,247,north,move_forward,217,north) transition(1,32,217,north,turn_right,218,east) transition(1,33,218,east,move_forward,188,north)`
	
	'ICE' vs 'local':
	
	Optimum found. Appears **correct**.
	
	'ICE' vs 'freight':
	
	Optimum found. Appears **correct**.
	
	'Local' vs 'freight':
	
	Optimum found. Appears **correct**.
	
	'Freight' vs 'freight':

	Optimum found. Appears **correct**. **Success.** Conclusion: speed **works**. There is no need to test it on more large instances apart from observing computation speed differences. But since at this stage it is not our priority, the proof of concept demonstrated above **suffices**.

INTUITION: if I want to write my own visualizer, ‘transition’ / ‘laggy_transition’ atoms are the best input - using ‘action’ would imply inference from the starting position (but would help other groups).

ACCELERATION

What am I to start with? The most reasonable is just to understand what acceleration is in terms of answer sets (and my implementation of speed) vs its standard definition.

Standard Definition	**Answer Sets Interpretation**
‘Acceleration is the rate at which the velocity of an object changes over time. It's a vector quantity, meaning it has both magnitude (how much the velocity is changing) and direction. In simpler terms, acceleration is how quickly an object speeds up, slows down, or changes direction.’	Given that speed, in my interpretation, is the **frequency of changing states**, acceleration is *the rate at which frequency of changing states changes over moments of time*.

Let’s switch back to the simplest encoding for this development.

INTUITION 1: create speed_range atom for every train depending on its default speed. For instance: an ICE can slow down to ½ and ⅓, a local can slow down to ⅓ and a freight’s speed can only be decreased to waiting (not necessary, but for instance).

Let’s read a little about how the speed of trains is regulated in a real network (using Mistral):

Q: In a real railroad network, what are the rules regarding acceleration of trains? I’m interested in: a. Whether there is a ‘cruise speed’ for every train b. Whether they are allowed to accelerate beyond this cruise speed (and how much of a potential speed margin there is). d. Who chooses the acceleration: the driver or an external control officer / system.

LLM (Mistral): ’*The rules regarding the acceleration of trains in a real railroad network can vary depending on the country, the type of train, and the specific railroad company. However, I can provide some general insights:*

a. Cruise Speed: Trains typically have a designated "cruise speed" or maximum operating speed. This speed is determined by various factors including the type of train, track conditions, and safety regulations. High-speed trains, for example, have a higher cruise speed compared to freight trains.
b. Accelerating Beyond Cruise Speed: Trains are generally not allowed to exceed their designated maximum operating speed. This speed limit is set to ensure the safety of the passengers, crew, and cargo, as well as to maintain the integrity of the railroad tracks and equipment. There is usually no speed margin beyond the cruise speed for regular operations.
c. Acceleration Control: The control of train acceleration and speed can vary. In many modern trains, especially those with advanced signaling and control systems, the speed and acceleration are managed by an automated system. This system can receive inputs from external control centers that monitor train positions and track conditions. However, the train driver also plays a crucial role in managing the speed and ensuring safe operation, especially in response to immediate conditions that automated systems may not account for.'

This is enough for our approximation:

a. Every train has its cruise speed.

b. There's typically no margin for speeding up, but we'll implement this for the local train just to show the possibility for acceleration.

c. The control is external - so it's not the train who 'decides' whether to slow down / speed up, but an external control system.

INTUITION 1 feels correct.

Let's assume I have these input facts for speed:

ICE: speed(0,1).
local: speed(1,2).
freight: speed(2,3).

Now, a normal rule must create, for each train, its possible speed range:

speed_range(ID, SPEED…3) :- speed(ID, SPEED).

Why 3? We choose 3 to be the slowest speed increment before full stopping in the system. This can be changed, but for an early prototype this rough discretization is enough.

Let's try to actually run this tiny piece in code:

```
Answer: 1
speed_range(0,1) speed_range(0,2) speed_range(0,3)
speed_range(1,2) speed_range(1,3)
speed_range(2,3)
```

All correct. Now, each train has its own set of atoms holding its potential speeds.

INTUITION 2: I'll need to use these atoms for generation with the choice rule (in the main generator of the program).

There is a core logic issue with acceleration:

```
action_moment(TRAIN, MOMENT) :- MOMENT = 0..MAX_TIME-1,
                                end(TRAIN, (_, _), MAX_TIME),
                                train(TRAIN),
                                speed(TRAIN, MOM_PER_POS),
                                R = MOMENT - (MOMENT / MOM_PER_POS) * MOM_PER_POS,
                                R == 0.

every_moment(TRAIN, 0..MAX_TIME-1) :- end(TRAIN, (_, _), MAX_TIME).

1 { action_transition(TRAIN, MOMENT, FROM_POS, IN_DIR, ACTION, TO_POS, OUT_DIR) : path(IN_DIR, FROM_POS, ACTION, OUT_DIR, TO_POS) } 1 :- train(TRAIN),
                                                                                                                                        action_moment(TRAIN, MOMENT).

intermediate_missing(TRAIN, MOMENT) :- every_moment(TRAIN, MOMENT), not action_moment(TRAIN, ACTION_MOMENT), MOMENT == ACTION_MOMENT.
```

The way we create speed now seems to make it unchangeable during generation.